DOCUMENTACIÓN DE PRUEBAS AUTOMATIZADAS

Aplicación Android: Sauce Labs My Demo App | Carrito de compras

Dato	Detalle
Proyecto	QAInnovationLab / MobileAutomationAppium
Repositorio personal	https://github.com/Armandina1984/MobileAutomationAppium
Rama publicada	automation-mobile-v2
Commit observado	4869c9e
Fecha del documento	14 de octubre de 2026
Estado	Publicado en GitHub; automatización aún en corrección

1. Objetivo y alcance

Automatizar, con Appium y Cucumber, la selección de productos en la galería de la aplicación Android My Demo App, la modificación de unidades y la verificación del carrito de compras. El alcance actual contiene tres combinaciones de producto y cantidad; no incluye pago, inicio de sesión ni compra finalizada.

2. Entorno y herramientas

Componente	Configuración conocida
Lenguaje	Java 21
Automatización móvil	Appium + UiAutomator2
BDD	Cucumber / Gherkin
Gestión de dependencias	Maven (pom.xml)
IDE	IntelliJ IDEA
Dispositivo	Samsung SM-A135F, Android 14
Aplicación	Sauce Labs My Demo App, APK mda-2.0.2-23.apk
Paquete Android	com.saucelabs.mydemoapp.android
Servidor Appium	http://127.0.0.1:4723/
Control de versiones	Git y GitHub

La dirección inalámbrica del dispositivo puede cambiar entre sesiones; por ello se omite como dato permanente de configuración,pero se uso en este caso ya que con cable no respondia.

3. Organización de archivos

src/test/java/com/nttdata/screens/CarritoComprasScreen.java
src/test/java/com/nttdata/steps/CarritoComprasSteps.java
src/test/java/com/nttdata/stepsdefinitions/CarritoComprasStepsDefs.java
src/test/resources/config/mydemoapp.properties
src/test/resources/features/CarritoCompras.feature
CarritoComprasScreen centraliza localizadores y operaciones de interfaz. CarritoComprasSteps coordina el flujo. CarritoComprasStepsDefs conecta los pasos Gherkin con Java. El archivo properties configura la sesión Android y el feature define los escenarios.

4. Historia de usuario y criterios de aceptación.

Como usuario de la tienda móvil, quiero seleccionar productos y agregarlos en cantidades específicas al carrito para comprobar que la compra refleja correctamente lo elegido.

CA01: Al ingresar, la galería debe mostrar el título Products y la lista de productos.

CA02: Al elegir un producto, el usuario debe poder establecer la cantidad indicada.

CA03: El carrito debe mostrar el producto elegido, su cantidad y el total de artículos correspondiente.

5. Escenarios de prueba en Gherkin

```
Feature: Validar carrito de compras en My Demo App
  Scenario Outline: Agregar productos al carrito de compras
    Given ingreso al aplicativo "mydemoapp"
    And valido que carguen correctamente los productos en la galeria
    When agrego <UNIDADES> del siguiente producto "<PRODUCTO>"
    Then valido el carrito de compra actualice correctamente

    Examples:
      | PRODUCTO                | UNIDADES |
      | Sauce Labs Backpack     | 1        |
      | Sauce Labs Bolt - T-Shirt | 1      |
      | Sauce Labs Bike Light   | 2        |
```

6. Matriz de casos de prueba

ID	Datos	Resultado esperado	Estado verificado
CP-MOB-01	Backpack / 1	Carrito muestra Backpack y 1 Item	No completado; error en localizador de cantidad
CP-MOB-02	Bolt - T-Shirt / 1	Carrito muestra el polo y 1 Item	No se dispone de ejecución exitosa
CP-MOB-03	Bike Light / 2	Carrito muestra la luz y 2 Items	No se dispone de ejecución exitosa

La aplicación utiliza el nombre Sauce Labs Bolt T-Shirt en determinados localizadores; el código normaliza la variante del feature Sauce Labs Bolt - T-Shirt.

7. Flujo automatizado implementado:

1. Inicializar Appium desde mydemoapp.properties y abrir la aplicación.

2. Abrir carrito y retirar productos preexistentes para aislar el escenario.

3. Regresar a Products y validar galería.

4. Seleccionar producto por nombre.

5. Leer y ajustar cantidad mediante controles + y −.

6. Agregar al carrito y verificar nombre, unidades y contador total.

8. Localizadores relevantes

Pantalla / control	resource-id
Galería / título	productTV
Galería / lista	productRV
Icono de carrito	cartRL
Carrito vacío / contenedor	noItemCL
Carrito vacío / mensaje	noItemTitleTV
Carrito vacío / botón Go Shopping	shoppingBt
Detalle / cantidad (pendiente de confirmar)	noTV
Detalle / aumentar	plusIV
Detalle / disminuir	minusIV
Detalle / agregar	cartBt
Carrito con productos / contenedor (pendiente de confirmar)	cartCL
Carrito / eliminar	removeBt
Carrito / total	itemsTV

Los IDs noItemCL y shoppingBt se comprobaron en el XML exportado de Appium Inspector. La validez de noTV en la pantalla de detalle y cartCL en el carrito con productos aún requiere inspección.

9. Evidencias y ejecución observada

Evidencia	Resultado registrado
Sesión Appium	Creada correctamente desde IntelliJ
Carrito vacío	Se abrió y se limpió correctamente
Galería	Se regresó y validó Products correctamente
Selección de producto	Se seleccionó Sauce Labs Backpack
Paso de cantidad	TimeoutException al localizar id/noTV, 15 segundos
Resultado integral	No hay evidencia de tres escenarios aprobados

Fragmentos relevantes del log de ejecución:

[QA] Carrito abierto
[QA] Carrito limpio
[QA] Regreso a la galeria
[QA] Galeria de productos cargada correctamente
[QA] Producto seleccionado: Sauce Labs Backpack
TimeoutException: waiting for visibility of element
By.id: com.saucelabs.mydemoapp.android:id/noTV
CarritoComprasScreen.seleccionarCantidad(...:121)

Se mostro en las siguientes capturas de las tres ejecuciones finales y actualizar los estados de CP-MOB-01 a CP-MOB-03.

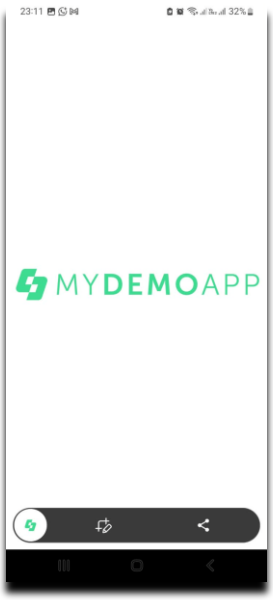


Imagen 1

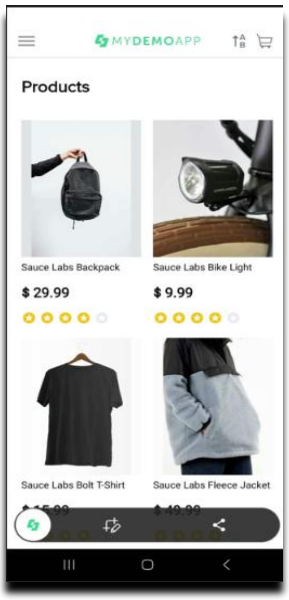


Imagen 2

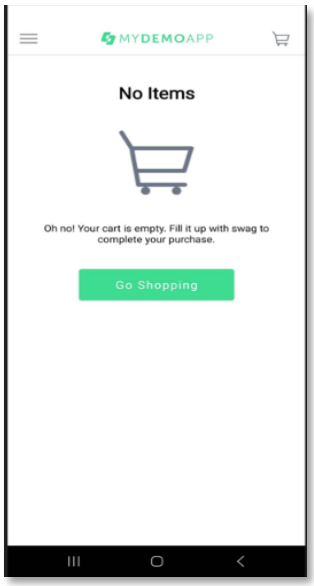


Imagen 3

10. Incidencias técnicas y seguimiento

Incidencia	Diagnóstico	Acción / situación
INC-01: apertura de carrito vacío	Se esperaba cartCL, pero la vista vacía utiliza noItemCL.	Corregido en abrirCarrito() para aceptar ambas vistas; la ejecución posterior avanzó.
INC-02: cantidad de producto	noTV no se encontró después de seleccionar Backpack.	Pendiente: inspeccionar XML de detalle y corregir localizador o transición.
INC-03: validación de carrito con productos	cartCL aún no verificado mediante XML de esa pantalla.	Pendiente: validar localizadores al superar INC-02.

11. Conclusiones y próximos pasos

El proyecto está estructurado, versionado y publicado en GitHub. La prueba llegó satisfactoriamente hasta la selección del primer producto, pero la validación del carrito no ha sido completada. No corresponde declarar aprobados los tres casos de prueba.

• Inspeccionar la pantalla de detalle de Backpack y confirmar el ID del contador de unidades.

• Corregir seleccionarCantidad() y volver a ejecutar el Scenario Outline completo.

• Verificar los localizadores y textos reales del carrito con productos.

• Queda pendiente crear un nuevo commit y subirlo al mismo repositorio una vez corregidas las pruebas.

Imagen Final al ejecutar el Carrito de compras.

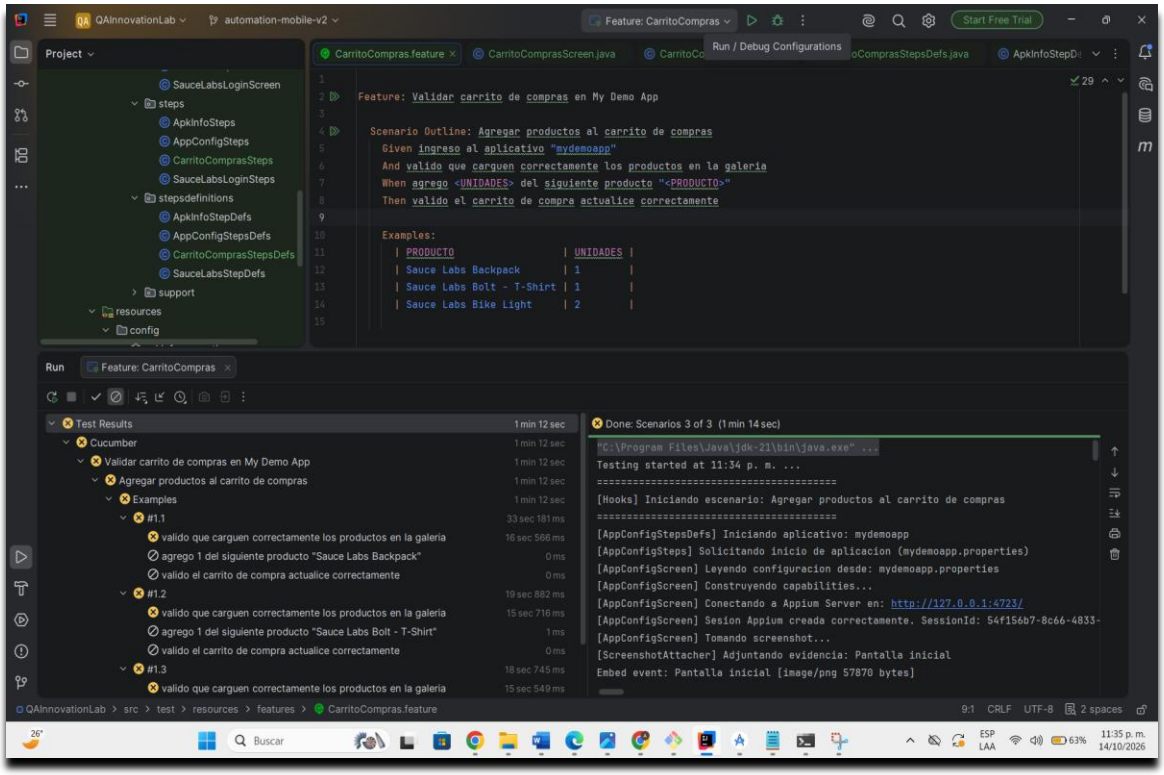


Imagen 4

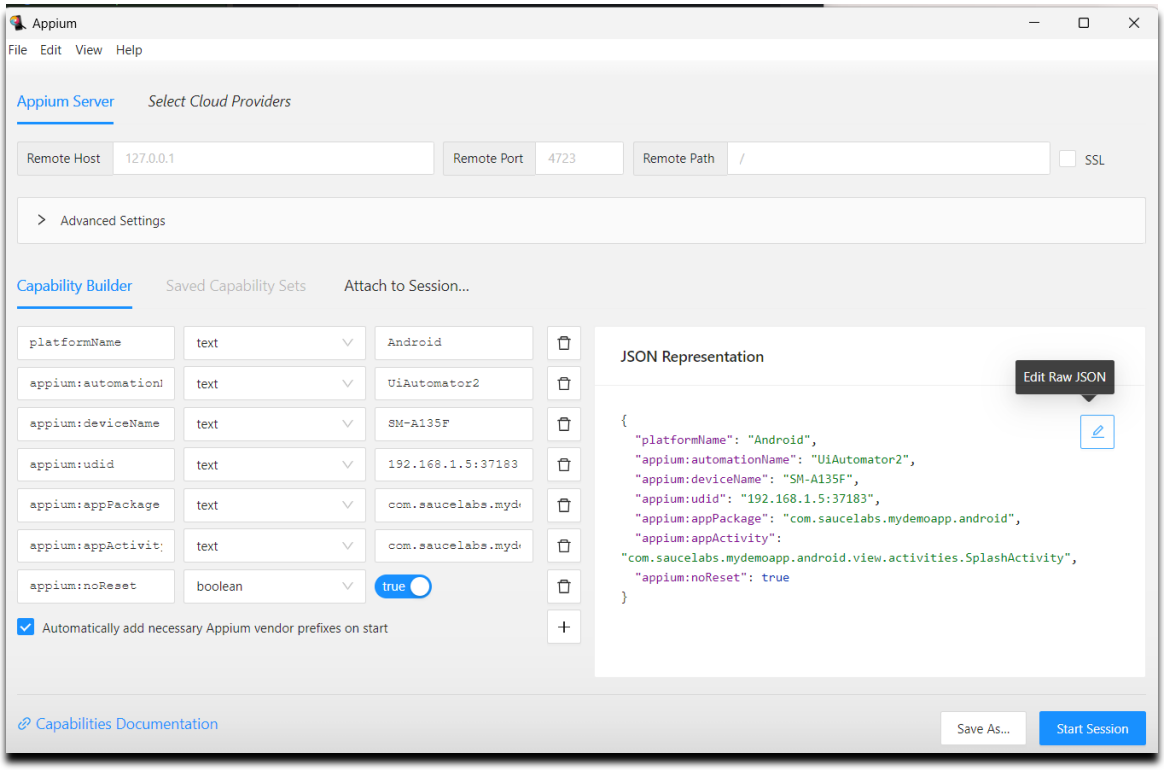


Imagen 5

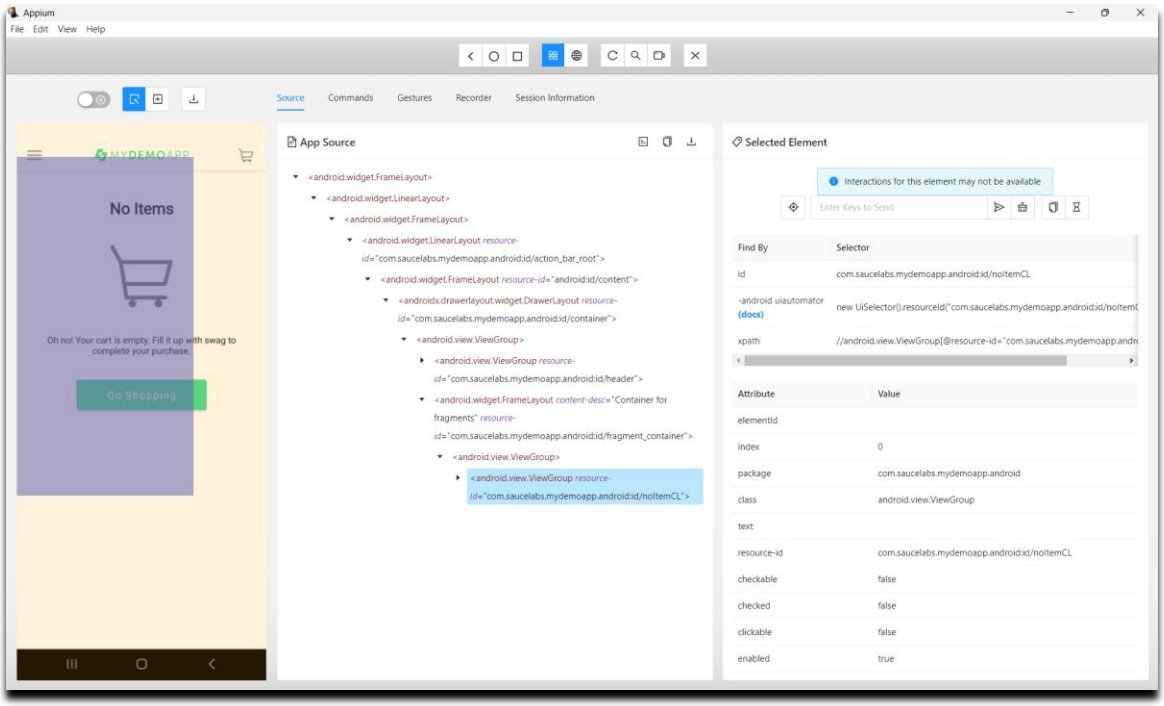


Imagen 6